

## ROTEIRO DE ESTUDO

### Semana 1

# Fundamentos do Paradigma Orientado a Objetos

Disciplina: Programação Orientada a Objetos

Aula teórica - abordagem conceitual com exemplos práticos de aplicação em Java

## 1. Paradigmas de programação e a ideia de organizar soluções

*A orientação a objetos é, antes de tudo, uma forma de pensar e estruturar uma solução de software.*

Um paradigma de programação pode ser entendido como uma maneira organizada de enxergar problemas e construir soluções computacionais. Ele fornece princípios para decidir quais elementos do problema serão representados, como as responsabilidades serão distribuídas e como as partes de um sistema irão interagir. Por isso, um paradigma não deve ser confundido com uma linguagem de programação. Linguagens são meios técnicos para implementar soluções; paradigmas são formas de raciocinar sobre essas soluções.

Em problemas muito simples, é possível pensar apenas em uma sequência de passos: receber uma informação, realizar um cálculo, tomar uma decisão e apresentar um resultado. À medida que os sistemas crescem, porém, surgem muitos dados, regras, situações de uso, usuários e dependências. Nesse cenário, a organização do raciocínio torna-se tão importante quanto a própria execução das operações. A Programação Orientada a Objetos (POO) surge como uma estratégia para lidar com essa complexidade por meio da identificação de elementos que possuem responsabilidades próprias e colaboram entre si.

A ideia central é aproximar a estrutura do software da maneira como descrevemos um domínio. Em um sistema acadêmico, por exemplo, é natural falar em aluno, disciplina, matrícula, turma e professor. Em um sistema de biblioteca, aparecem livro, exemplar, usuário, empréstimo e reserva. Esses nomes, contudo, não se transformam automaticamente em classes. Eles funcionam como candidatos de análise: o projetista precisa verificar se cada elemento tem relevância para o problema, se possui informações próprias e se desempenha algum papel na solução.

A POO também procura reduzir a concentração de decisões em uma única parte do sistema. Em vez de imaginar um grande procedimento que conhece todos os dados e executa todas as regras, distribuem-se responsabilidades entre componentes. Um pedido pode ser responsável por calcular seu total; um item pode conhecer seu preço e quantidade; um cliente pode manter seus próprios dados cadastrais. Essa distribuição favorece a compreensão do sistema porque cada elemento passa a ter um papel mais definido.

Outro ponto importante é que a orientação a objetos não significa copiar literalmente o mundo real. Um software é um modelo construído para atender objetivos específicos. Um objeto de software chamado "Conta", por exemplo, pode representar apenas as características necessárias a um sistema financeiro, e não todos os detalhes de uma conta existente no mundo real. A modelagem seleciona o que é relevante, omite o que não contribui para a solução e organiza as informações conforme as necessidades do sistema.

A motivação para estudar POO, portanto, não está somente em aprender termos como classe e objeto. O objetivo é desenvolver uma forma de decompor problemas, reconhecer responsabilidades, representar informações de maneira coerente e compreender como partes independentes cooperam para produzir um comportamento mais amplo. Esses fundamentos serão retomados em todos os demais conteúdos da disciplina.

### Ponto-chave

Programação Orientada a Objetos não é sinônimo de uma linguagem específica. É um paradigma: um conjunto de conceitos e princípios usados para analisar, modelar e organizar soluções de software.

## 2. Visão procedural e visão orientada a objetos

*Comparar paradigmas ajuda a perceber que a principal diferença está no foco usado para decompor o problema.*

Uma abordagem procedural organiza a solução principalmente em torno de procedimentos, rotinas ou etapas de processamento. O raciocínio costuma partir da pergunta: “quais passos precisam ser executados para obter o resultado?”. Dados são fornecidos a esses procedimentos, que os consultam e modificam conforme a necessidade. Essa forma de pensar é adequada para muitos problemas e não deve ser tratada como incorreta ou ultrapassada. Em tarefas com fluxo bem definido e pouca complexidade estrutural, ela pode ser simples e eficiente.

Na orientação a objetos, a pergunta inicial tende a mudar: “quais elementos participam do problema e quais responsabilidades cada um deve assumir?”. O foco deixa de estar apenas na sequência global de passos e passa a considerar unidades que reúnem informações e comportamentos relacionados. Em vez de uma rotina central saber tudo sobre todos os dados, procura-se atribuir a cada objeto aquilo que ele deve conhecer e aquilo que deve fazer.

Considere conceitualmente um sistema de vendas. Em uma visão fortemente procedural, poderíamos imaginar um processo chamado “fechar venda” que recebe os dados do cliente, percorre os itens, calcula subtotais, aplica descontos, calcula o total e registra o pagamento. Em uma visão orientada a objetos, essas responsabilidades podem ser distribuídas: o item conhece quantidade e preço; o pedido mantém seus itens e calcula o total; uma política de desconto determina condições; o pagamento registra a forma utilizada. O resultado final pode ser o mesmo, mas a organização interna é diferente.

Essa mudança de foco traz uma consequência importante: a manutenção do sistema tende a ser pensada localmente. Se a regra de cálculo do total pertence ao pedido, alterações nessa regra devem ser tratadas no componente que possui essa responsabilidade. Quando responsabilidades estão espalhadas de forma inadequada, uma mudança simples pode exigir intervenções em muitos lugares e aumentar a possibilidade de inconsistências.

Também é importante evitar a falsa ideia de que todo sistema orientado a objetos elimina sequências de passos. Objetos executam operações e colaboram em fluxos; portanto, procedimentos continuam existindo. A diferença é que esses procedimentos ficam associados a responsabilidades e contextos. O paradigma orientado a objetos não apaga o fluxo de execução, mas procura organizar melhor quem executa cada parte desse fluxo.

Outra distinção útil está na relação entre dados e operações. Em uma decomposição procedural, frequentemente se pensa primeiro nos dados e em funções que os manipulam. Na POO, procura-se manter próximos os dados de um conceito e os comportamentos que dependem desses dados. Essa proximidade sustenta ideias que serão estudadas posteriormente, como encapsulamento, coesão e proteção do estado interno.

| Aspecto              | Visão procedural                  | Visão orientada a objetos                   |
|----------------------|-----------------------------------|---------------------------------------------|
| Foco inicial         | Sequência de operações            | Entidades, responsabilidades e colaboração  |
| Pergunta orientadora | Quais passos executar?            | Quem deve saber e fazer o quê?              |
| Organização          | Procedimentos + dados             | Objetos com estado e comportamento          |
| Manutenção           | Pode concentrar regras em rotinas | Busca localizar regras junto ao responsável |

### Atenção

A comparação não estabelece que um paradigma é sempre superior ao outro. O critério central é compreender como cada abordagem organiza a solução e por que a orientação a objetos é especialmente útil quando há muitos conceitos, regras e interações.

### 3. Objeto: identidade, estado e comportamento

*O objeto é a unidade conceitual básica da orientação a objetos.*

Em POO, um objeto representa uma entidade relevante para o sistema. Essa entidade pode corresponder a algo concreto, como um livro ou um veículo, ou a algo abstrato, como uma matrícula, uma reserva, uma autorização ou um pagamento. O que caracteriza um objeto não é ser uma “coisa física”, mas possuir significado no domínio e participar da solução com informações e comportamentos próprios.

Três ideias ajudam a compreender um objeto: identidade, estado e comportamento. A identidade permite distinguir um objeto de outro, mesmo quando ambos apresentam valores semelhantes. Duas contas podem ter o mesmo saldo, mas continuam sendo contas diferentes. Dois exemplares do mesmo livro podem compartilhar título e edição, mas cada exemplar possui uma identidade própria para fins de empréstimo. A identidade responde à necessidade de reconhecer que cada instância é individual no contexto do sistema.

O estado corresponde ao conjunto de valores que descrevem a situação atual do objeto em determinado momento. Um pedido pode estar “aberto”, “pago” ou “cancelado”; uma reserva pode ter data, responsável e situação; um produto pode ter preço vigente e quantidade disponível. O estado não é apenas uma lista de informações: ele representa condições que podem influenciar quais comportamentos são possíveis. Um empréstimo já encerrado, por exemplo, não deveria ser renovado da mesma forma que um empréstimo ativo.

O comportamento representa aquilo que o objeto é capaz de realizar ou responder. Em uma modelagem conceitual, comportamentos podem ser descritos por verbos ou ações: calcular total, cancelar reserva, registrar pagamento, renovar empréstimo, alterar endereço. Um bom comportamento está relacionado à responsabilidade do próprio objeto e utiliza as informações que ele conhece. Essa relação entre estado e comportamento evita que outros componentes precisem conhecer detalhes desnecessários para executar regras internas.

É útil perceber que estado e comportamento estão ligados. Uma operação pode modificar o estado do objeto; por outro lado, o estado pode limitar ou condicionar a execução de uma operação. Quando um pedido é cancelado, por exemplo, seu estado muda. Depois disso, certas ações podem deixar de fazer sentido. Essa associação favorece a construção de modelos consistentes, pois as regras deixam de ser tratadas como fatos isolados e passam a fazer parte do ciclo de vida dos objetos.

Nem toda informação relacionada a uma entidade deve necessariamente pertencer ao seu estado. O critério é a responsabilidade. Em um sistema acadêmico, a idade de um aluno pode ser calculada a partir da data de nascimento; armazenar simultaneamente idade e data pode criar redundância, pois a idade muda com o tempo. Da mesma forma, o total de um pedido pode ser derivado de seus itens em vez de ser mantido como um valor independente sem controle. A modelagem precisa considerar quais informações são fundamentais e quais podem ser obtidas por comportamento.

Também não se deve reduzir um objeto a um “registro com dados”. Na orientação a objetos, um objeto relevante combina informações com responsabilidades. Se uma entidade só armazena valores e todas as decisões ficam concentradas fora dela, o modelo pode perder parte dos benefícios da orientação a objetos. Ao longo da disciplina, essa ideia será aprofundada por conceitos como encapsulamento e coesão.

#### Síntese conceitual

Identidade distingue o objeto; estado descreve sua situação atual; comportamento expressa o que ele pode fazer. Esses três elementos devem ser analisados em conjunto.

## 4. Classe e objeto: conceito geral e ocorrência específica

*Classes organizam características comuns; objetos representam ocorrências concretas desse conceito no sistema.*

Uma classe é uma descrição conceitual usada para representar um conjunto de objetos que compartilham uma estrutura e um conjunto de comportamentos esperados. Ela define o tipo de informação relevante para aquele conceito e as responsabilidades associadas. Um objeto, por sua vez, é uma ocorrência específica dessa classe, com identidade e valores próprios.

Considere a ideia de “Livro” em um sistema. A classe Livro poderia indicar que livros possuem título, edição e identificação bibliográfica, além de comportamentos relacionados ao domínio. Já um objeto específico representaria uma obra determinada, com valores particulares. Se o sistema também controla cópias físicas, talvez seja necessário distinguir Livro de Exemplar: Livro representaria a obra, enquanto Exemplar representaria cada unidade física disponível. Esse exemplo mostra que escolher classes é uma decisão de modelagem, e não apenas uma tarefa de localizar substantivos em um texto.

Uma classe não deve ser entendida apenas como um formulário de preenchimento. Ela expressa uma ideia mais ampla: quais informações caracterizam aquele tipo de objeto, quais operações fazem sentido para ele e quais regras precisam ser preservadas. Os atributos descrevem aspectos do estado; os comportamentos descrevem ações ou respostas. Ambos devem guardar relação com a responsabilidade atribuída à classe.

A separação entre classe e objeto permite falar em características gerais sem perder a individualidade das instâncias. A classe “Aluno” pode definir que alunos possuem nome, identificador institucional e situação acadêmica. Entretanto, cada aluno possui valores próprios e sua própria identidade. Alterar o estado de um objeto não significa alterar automaticamente todos os demais objetos da mesma classe.

Ao modelar classes, é necessário evitar extremos. Uma classe excessivamente genérica, como “Coisa”, “Dados” ou “GerenciadorDeTudo”, tende a reunir responsabilidades sem relação clara. Por outro lado, criar uma classe para cada pequeno detalhe pode fragmentar o modelo e dificultar sua compreensão. O objetivo é encontrar unidades conceituais coesas, que representem algo significativo no domínio e tenham uma razão clara para existir.

Outro cuidado está em distinguir “classe” de “categoria informal”. Nem toda classificação do mundo real precisa se tornar uma classe no software. O modelo deve responder às necessidades do sistema. Em um aplicativo de entregas, por exemplo, a cor dos olhos do cliente provavelmente não possui relevância. Já endereço de entrega, contato e histórico de pedidos podem ser importantes. A classe deve incluir apenas aquilo que contribui para os objetivos do sistema.

A qualidade dessa modelagem depende da capacidade de justificar cada classe. Uma justificativa simples pode responder a três pontos: o que essa classe representa; quais informações precisa conhecer; quais responsabilidades deve cumprir. Se não houver respostas claras, é possível que a classe esteja mal definida ou que o conceito deva ser representado de outra forma.

### Regra prática

Classe representa um conceito; objeto representa uma instância específica desse conceito. A existência de uma classe deve ser justificada pelas necessidades do domínio e pelas responsabilidades que ela assume.

## 5. Responsabilidades, mensagens e colaboração entre objetos

*Um sistema orientado a objetos é formado por participantes que cooperam, e não por objetos isolados.*

A noção de responsabilidade é central para a orientação a objetos. Responsabilidade significa aquilo que um objeto deve conhecer ou fazer dentro do sistema. Um pedido pode conhecer seus itens e sua situação; pode também calcular o próprio total. Um cliente pode conhecer seus dados de contato. Um pagamento pode registrar valor, data e situação. A qualidade da solução depende de atribuir cada responsabilidade ao componente mais apropriado.

Quando um objeto precisa que outro realize uma ação, ocorre uma colaboração. Conceitualmente, dizemos que um objeto envia uma mensagem ou solicita um comportamento a outro. Não é necessário associar essa ideia a uma sintaxe de linguagem. O importante é compreender a dinâmica: um participante possui uma necessidade e pede que outro, responsável por determinada tarefa, execute sua parte.

Imagine um processo de empréstimo em uma biblioteca. O usuário solicita um exemplar. O empréstimo registra datas e situação. O exemplar precisa estar disponível. O sistema pode ainda consultar regras de limite de empréstimos. A solução não precisa concentrar todas essas decisões em um único “superobjeto”. Em vez disso, pode distribuir as tarefas conforme o conhecimento de cada participante. Essa distribuição permite que o modelo reflita melhor as responsabilidades do domínio.

Uma regra útil para avaliar responsabilidades é observar se um objeto está tomando decisões com base em informações que pertencem essencialmente a outro. Se, para calcular o total de um pedido, uma parte externa precisa coletar preço e quantidade de cada item e reproduzir todas as regras de cálculo, talvez o comportamento esteja no lugar inadequado. Pode ser mais coerente solicitar ao próprio pedido que forneça seu total, deixando os detalhes de cálculo próximos dos objetos que conhecem os dados.

Essa maneira de pensar favorece modelos com maior coesão. Coesão significa, em termos iniciais, manter juntas responsabilidades relacionadas. Uma classe coesa tem um propósito compreensível. Quando uma classe acumula tarefas de cadastro, cálculo, autenticação, impressão, envio de mensagens e controle de estoque sem uma justificativa conceitual, torna-se difícil entender, testar e modificar suas regras.

A colaboração também exige equilíbrio. Objetos precisam se comunicar, mas dependências excessivas podem tornar o sistema rígido. Um objeto não deve conhecer detalhes internos de todos os demais. O ideal é que cada participante exponha apenas os comportamentos necessários para a cooperação. Essa ideia será aprofundada mais adiante com encapsulamento e interfaces, mas já pode ser percebida na primeira semana como um princípio de organização.

Ao analisar um cenário, portanto, não basta listar objetos. É necessário identificar quem inicia uma interação, quem possui a informação necessária, quem deve executar a regra e quais mudanças de estado podem ocorrer. Essa análise transforma uma lista estática de entidades em um modelo de colaboração orientado a objetos.

### **Critério de qualidade**

Uma boa distribuição de responsabilidades permite explicar o sistema com frases do tipo: “este objeto sabe isso”, “aquele objeto faz aquilo” e “eles colaboram desta forma”, sem depender de um componente central que conheça tudo.

## 6. Objetos do mundo real e objetos de software: semelhanças e limites

*Modelar não é reproduzir a realidade em todos os detalhes; é construir uma representação útil para um objetivo.*

A orientação a objetos utiliza frequentemente exemplos do mundo real porque eles facilitam a compreensão inicial. Carros, contas, livros, pessoas e pedidos ajudam a ilustrar identidade, estado e comportamento. Entretanto, um sistema de software não é uma cópia da realidade. Ele é um modelo seletivo, criado para atender requisitos específicos. Essa distinção é essencial para evitar modelos excessivamente literais.

Considere uma pessoa. No mundo real, uma pessoa possui inúmeras características físicas, sociais e históricas. Em um sistema escolar, porém, talvez sejam relevantes apenas nome, matrícula, curso e situação acadêmica. Em um sistema de saúde, outras informações serão necessárias. O mesmo conceito do mundo real pode gerar modelos diferentes porque cada sistema tem objetivos, regras e fronteiras diferentes.

Também existem objetos de software que não possuem equivalente físico evidente. Uma reserva, uma sessão de autenticação, uma política de desconto, uma autorização ou um período letivo são conceitos abstratos, mas podem ter identidade, estado e comportamento. Portanto, a pergunta adequada não é “isso existe fisicamente?”, mas “isso é um conceito importante para a solução e possui responsabilidades próprias?”.

O contexto do problema define o que entra ou não no modelo. Essa fronteira impede que o sistema tente representar tudo. Em uma biblioteca, por exemplo, pode ser importante saber se um exemplar está disponível, emprestado ou em manutenção. Talvez não seja necessário representar a cor da capa ou o tipo de papel. Um atributo só deve aparecer se tiver utilidade para regras, decisões, registros ou funcionalidades previstas.

Essa seleção de características é uma forma de abstração, conceito que será aprofundado na semana seguinte. Nesta primeira etapa, é suficiente compreender que abstrair significa concentrar atenção no que é relevante e ignorar detalhes que não ajudam a resolver o problema. A abstração não empobrece o modelo; ela o torna manejável.

Outro erro frequente é transformar cada substantivo de uma descrição em classe. Textos de requisitos contêm pessoas, locais, datas, valores, ações e qualificadores, mas nem todos precisam ser classes. “Data”, por exemplo, pode ser apenas uma informação associada a um evento; “biblioteca” pode ser o contexto do sistema, não necessariamente um objeto relevante; “relatório” pode ser um resultado, e não uma entidade duradoura. O projetista deve justificar a necessidade de cada conceito.

Por fim, um bom modelo precisa permanecer compreensível para pessoas que conhecem o domínio. Os nomes devem refletir conceitos reconhecíveis e as responsabilidades devem fazer sentido. A proximidade com a linguagem do domínio facilita a comunicação entre equipe técnica e usuários, reduz ambiguidades e ajuda a validar se a solução representa corretamente as regras essenciais.

### Princípio de modelagem

Um modelo orientado a objetos deve representar o que é relevante para o objetivo do sistema, e não tudo o que existe no mundo real. Relevância, responsabilidade e clareza são critérios mais importantes que semelhança literal.

## 7. Aplicação integrada dos fundamentos e erros conceituais frequentes

*A compreensão dos conceitos iniciais deve permitir analisar um problema simples antes de qualquer implementação.*

Para aplicar os fundamentos da primeira semana, considere um cenário de clínica. Pacientes solicitam consultas; médicos possuem especialidades; consultas são agendadas para uma data e horário e podem ser confirmadas ou canceladas. O primeiro passo é identificar conceitos relevantes: Paciente, Médico e Consulta são candidatos naturais. “Agendamento” pode ser representado pela própria Consulta ou por outro conceito, dependendo das regras do sistema. A decisão deve ser baseada na responsabilidade, e não apenas no vocabulário usado na descrição.

Em seguida, analisamos identidade, estado e comportamento. Cada consulta precisa ser distinguida das demais; portanto, possui identidade. Seu estado pode envolver data, horário, paciente, médico e situação. Seus comportamentos podem incluir confirmar, cancelar ou reagendar, desde que essas ações pertençam à responsabilidade atribuída à consulta. O paciente também possui identidade e estado próprios, enquanto o médico pode manter informações necessárias ao atendimento.

Depois, observamos as colaborações. Para agendar uma consulta, é necessário associar paciente e médico em um horário válido. Para cancelar, o objeto responsável precisa alterar a situação da consulta e talvez liberar o horário. Mesmo sem detalhar uma implementação, já é possível discutir quem deve conhecer cada informação e quem deve executar cada ação. Esse exercício é justamente o raciocínio orientado a objetos.

Alguns erros conceituais são comuns nessa etapa. O primeiro é confundir classe com objeto: “Paciente” é um conceito geral, enquanto uma pessoa específica cadastrada no sistema é um objeto. O segundo é tratar atributo como sinônimo de qualquer informação possível. Um atributo deve ser relevante para o estado e para as necessidades do domínio. O terceiro é criar classes apenas para guardar dados, deslocando todos os comportamentos para um controlador central. Isso enfraquece a ideia de responsabilidade.

Outro erro é acreditar que objetos precisam representar apenas coisas concretas. Como visto, conceitos abstratos também podem ser objetos. Da mesma forma, não se deve pensar que todo comportamento precisa alterar estado. Alguns comportamentos apenas consultam ou calculam informações. Um pedido pode calcular seu total sem necessariamente modificar seus dados; um catálogo pode localizar itens; uma reserva pode informar se está ativa.

Também é inadequado imaginar que orientação a objetos significa criar o maior número possível de classes. Um bom modelo não é o mais extenso, mas o mais coerente. Classes devem existir quando representam conceitos significativos e possuem responsabilidades claras. A simplicidade é uma qualidade importante, especialmente nos primeiros modelos.

Ao final desta semana, espera-se que o estudante consiga ler um pequeno problema e produzir uma descrição inicial orientada a objetos: identificar candidatos a objetos, distinguir classes de instâncias, descrever estado e comportamento e explicar como os participantes colaboram. Não se espera ainda um modelo definitivo. O objetivo é formar a base conceitual necessária para aprofundar abstração, atributos, métodos, encapsulamento, associações, herança e polimorfismo nas semanas seguintes.

### Checklist de domínio da Semana 1

- Explicar, com palavras próprias, o que é um paradigma de programação.
- Distinguir a visão procedural da visão orientada a objetos sem afirmar que uma é sempre superior à outra.
- Definir objeto a partir de identidade, estado e comportamento.
- Diferenciar classe de objeto/instância.
- Identificar responsabilidades e colaborações em um cenário simples.
- Justificar por que nem todo elemento do mundo real deve ser representado no software.

#### Síntese final

A orientação a objetos organiza soluções em torno de entidades relevantes que possuem identidade, estado e comportamento, assumem responsabilidades e colaboram. Classes descrevem conceitos comuns; objetos são ocorrências específicas. A modelagem é seletiva e deve refletir os objetivos do sistema.

## PONTE ENTRE TEORIA, UML E IMPLEMENTAÇÃO

## Exemplos práticos em Java e diagramas UML

Os exemplos a seguir não substituem o estudo conceitual da Programação Orientada a Objetos. Seu objetivo é mostrar como ideias como classe, objeto, estado, comportamento, responsabilidade e colaboração podem aparecer em uma implementação Java. Elementos como modificadores de acesso, construtores, coleções e relacionamentos serão aprofundados em semanas posteriores.

### Como ler os exemplos

Primeiro observe o problema e o diagrama; depois identifique no código qual trecho representa a classe, quais valores formam o estado do objeto e quais métodos expressam comportamentos. A prioridade é compreender a relação entre modelo e implementação.

### Exemplo prático 1 - Classe, objeto, estado e comportamento

Situação: um sistema simples precisa representar livros e controlar se um livro está disponível para empréstimo. A classe Livro descreve a estrutura comum. Cada livro criado no programa é um objeto com seus próprios valores.

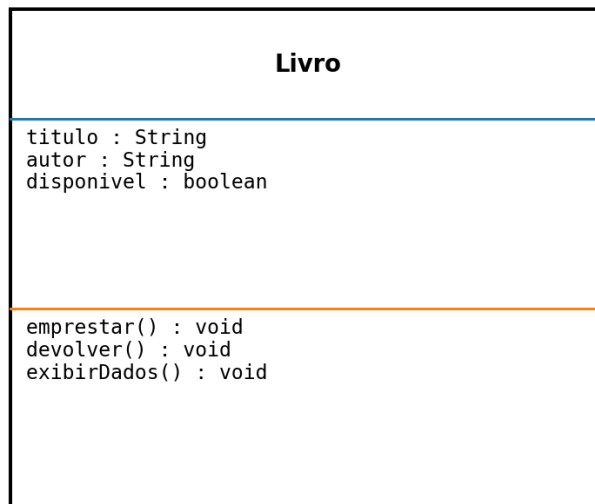


Diagrama de classe: estrutura conceitual da classe Livro

Figura 1 - Diagrama de classe UML simplificado para Livro.

#### Livro.java

```

public class Livro {
    public String titulo;
    public String autor;
    public boolean disponivel;

    public void emprestar() {
        if (disponivel) {
            disponivel = false;
            System.out.println("Empréstimo realizado.");
        } else {
            System.out.println("Livro indisponível.");
        }
    }

    public void devolver() {
        disponivel = true;
        System.out.println("Livro devolvido.");
    }

    public void exibirDados() {
        System.out.println(titulo + " - " + autor);
        System.out.println("Disponível: " + disponivel);
    }
}
  
```

```

}
Principal.java

public class Principal {
    public class Main {
        public static void main(String[] args) {
            Livro livro1 = new Livro();

            livro1.titulo = "POO na Prática";
            livro1.autor = "Ana Souza";
            livro1.disponivel = true;

            livro1.exibirDados();
            livro1.emprestar();
            livro1.exibirDados();
        }
    }
}

```

Leitura orientada do exemplo:

- Livro é a classe: ela descreve quais informações e comportamentos os objetos desse tipo possuem.
- livro1 é um objeto (instância) criado a partir da classe Livro.
- titulo, autor e disponivel compõem o estado do objeto.
- emprestar(), devolver() e exibirDados() representam comportamentos.
- Ao executar emprestar(), o comportamento altera o estado: disponivel passa de true para false.

### Observação didática

Os atributos aparecem sem modificadores de acesso para manter o primeiro exemplo visualmente simples. Encapsulamento e visibilidade serão estudados formalmente mais adiante; em aplicações reais, esses aspectos devem ser tratados de forma adequada.

### Exemplo de diagrama de objeto

O diagrama de classe mostra a estrutura geral do conceito Livro. Já o diagrama de objeto representa uma instância em um determinado momento. A figura abaixo corresponde ao objeto livro1 logo após a atribuição inicial de seus valores.

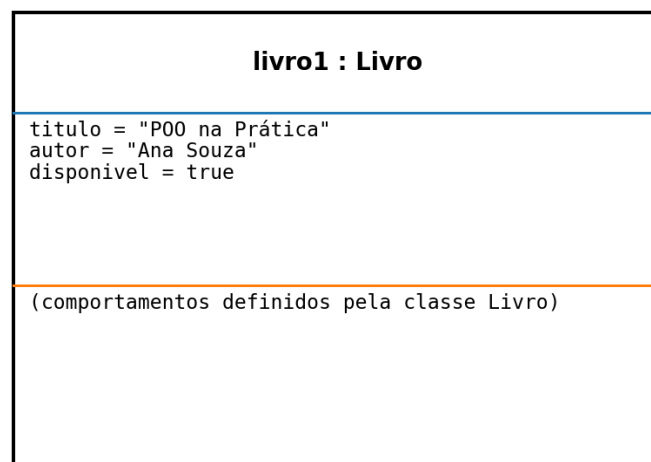


Diagrama de objeto: uma instância concreta e seu estado atual

Figura 2 – Diagrama de objeto para a instância livro1 : Livro.

Depois da chamada livro1.emprestar(), a identidade do objeto continua sendo a mesma, mas seu estado é alterado: o valor de disponivel passa a ser false. Esse é um ponto importante da POO: o objeto permanece sendo a mesma instância enquanto seus valores podem mudar ao longo do tempo.

## Exemplo prático 2 – Estado condicionando comportamentos

Situação: uma conta bancária possui um saldo. O comportamento sacar deve considerar o estado atual da conta, pois um saque não deve ser realizado quando o valor solicitado ultrapassa o saldo disponível.

### ContaBancária.java

```
public class ContaBancaria {
    public String titular;
    public double saldo;

    public void depositar(double valor) {
        if (valor > 0) {
            saldo = saldo + valor;
        }
    }

    public void sacar(double valor) {
        if (valor > 0 && valor <= saldo) {
            saldo = saldo - valor;
            System.out.println("Saque realizado.");
        } else {
            System.out.println("Saque não permitido.");
        }
    }

    public void mostrarSaldo() {
        System.out.println("Titular: " + titular);
        System.out.println("Saldo: R$ " + saldo);
    }
}
```

### Principal.java

```
public class Principal {
    public static void main(String[] args) {
        ContaBancaria conta = new ContaBancaria();

        conta.titular = "Carlos";
        conta.saldo = 500.00;

        conta.depositar(200.00);
        conta.sacar(100.00);
        conta.mostrarSaldo();
    }
}
```

O exemplo evidencia uma relação direta entre estado e comportamento:

- saldo representa parte do estado atual da conta.
- depositar() pode modificar o saldo, desde que o valor seja válido.
- sacar() consulta o estado antes de decidir se pode alterá-lo.
- mostrarSaldo() consulta o estado sem modificá-lo.

### Pergunta para reflexão

Por que seria inadequado permitir que qualquer parte do sistema alterasse o saldo livremente, sem respeitar as regras de depósito e saque? Essa pergunta antecipa a importância do encapsulamento.

## Exemplo prático 3 - Responsabilidade e colaboração entre objetos

Situação: em uma biblioteca, um empréstimo relaciona um usuário a um livro. A classe Empréstimo coordena a operação, mas solicita ao próprio Livro que realize a mudança de disponibilidade. Assim, as responsabilidades são distribuídas.

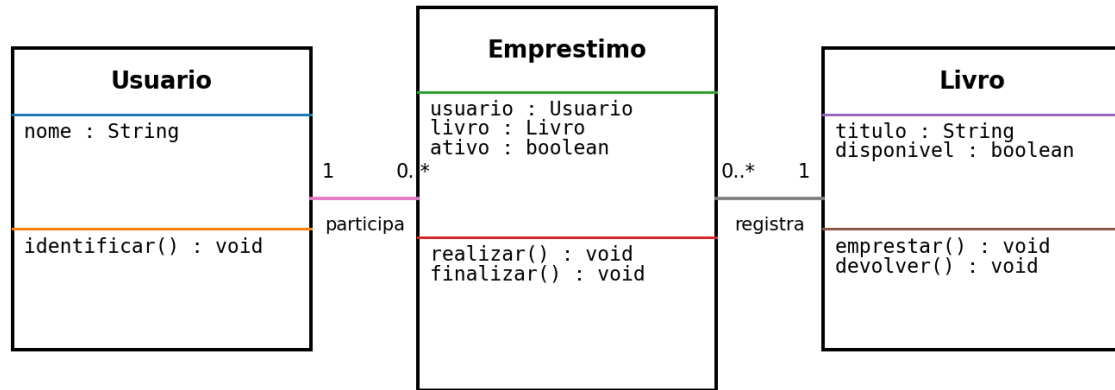


Diagrama de classes simplificado: colaboração entre Usuario, Emprestimo e Livro

Figura 3 - Diagrama de classes simplificado para uma operação de empréstimo.

#### Usuario.java

```
public class Usuario {
    String nome;

    public void identificar() {
        System.out.println("Usuário: " + nome);
    }
}
```

```
public class Livro {
    public String titulo;
    public boolean disponivel = true;

    public void emprestar() {
        disponivel = false;
    }

    public void devolver() {
        disponivel = true;
    }
}
```

#### Emprestimo.java

```
public class Emprestimo {
    public Usuario usuario;
    public Livro livro;
    public boolean ativo;

    public void realizar() {
        if (livro.disponivel) {
            livro.emprestar();
            ativo = true;
            System.out.println("Empréstimo realizado para " + usuario.nome);
        } else {
            System.out.println("Livro indisponível.");
        }
    }

    public void finalizar() {
        if (ativo) {
            livro.devolver();
            ativo = false;
            System.out.println("Empréstimo finalizado.");
        }
    }
}
```

#### BibliotecaApp.java

```
public class BibliotecaApp {
    public static void main(String[] args) {
        Usuario usuario = new Usuario();
    }
}
```

```

    usuario.nome = "Mariana";

    Livro livro = new Livro();
    livro.titulo = "Fundamentos de POO";

    Emprestimo emprestimo = new Emprestimo();
    emprestimo.usuario = usuario;
    emprestimo.livro = livro;

    emprestimo.realizar();
    emprestimo.finalizar();
}
}

```

### Análise da colaboração:

- Usuario representa quem participa do empréstimo.
- Livro conhece seu título e sua disponibilidade, além de saber alterar essa disponibilidade.
- Emprestimo registra a relação entre Usuario e Livro e controla se a operação está ativa.
- Ao realizar o empréstimo, Emprestimo solicita a Livro que execute emprestar(); há colaboração entre objetos.
- O código é deliberadamente simples. Associação, encapsulamento, construtores e outras decisões de projeto serão aprofundados posteriormente.

### Relacionando teoria, UML e Java

| Conceito teórico  | Representação UML                           | Representação no exemplo Java              |
|-------------------|---------------------------------------------|--------------------------------------------|
| Classe            | Retângulo com nome, atributos e operações   | class Livro { ... }                        |
| Objeto            | Instância, como livro1 : Livro              | Livro livro1 = new Livro();                |
| Estado            | Valores dos atributos/slots                 | titulo, autor, disponivel, saldo           |
| Comportamento     | Operações da classe                         | emprestar(), sacar(), devolver()           |
| Colaboração       | Associações entre classes/objetos           | emprestimo chama livro.emprestar()         |
| Mudança de estado | Valor do objeto se altera ao longo do tempo | disponivel = false; saldo = saldo - valor; |

### Exercício prático sugerido

Sem copiar diretamente os exemplos, modele e implemente em Java uma classe ReservaSala com, no mínimo, as informações sala, responsavel e ativa. Crie comportamentos reservar(), cancelar() e exhibirSituacao(). Depois, desenhe um diagrama de classe e um diagrama de objeto correspondente a uma reserva concreta. O objetivo é verificar se o estudante consegue passar do conceito para a representação UML e, em seguida, para uma implementação simples.

### Resultado esperado

O estudante deve conseguir explicar oralmente onde aparecem, em sua solução, a classe, o objeto, o estado, os comportamentos e a mudança de estado provocada por uma operação.

## ATIVIDADES DE FIXAÇÃO

**Questões - Semana 1****1. Em relação ao conceito de paradigma de programação, assinale a alternativa correta.**

- A) É uma linguagem específica utilizada obrigatoriamente para implementar sistemas orientados a objetos.
- B) É uma forma de organizar o raciocínio e estruturar soluções computacionais segundo determinados princípios.
- C) É um conjunto de comandos usados exclusivamente para criar interfaces gráficas.
- D) É uma técnica voltada apenas para representar dados, sem considerar comportamentos.

**2. Qual alternativa expressa melhor uma diferença entre a visão procedural e a visão orientada a objetos?**

- A) A visão procedural não utiliza dados, enquanto a orientação a objetos utiliza apenas dados.
- B) A visão procedural se concentra somente em interfaces, enquanto a orientação a objetos se concentra somente em bancos de dados.
- C) A visão procedural tende a organizar a solução em torno de passos e rotinas, enquanto a orientação a objetos enfatiza entidades, responsabilidades e colaboração.
- D) A orientação a objetos elimina qualquer sequência de operações e, por isso, não possui fluxo de execução.

**3. Na orientação a objetos, o estado de um objeto corresponde:**

- A) Ao conjunto de valores que descreve sua situação em determinado momento.
- B) Ao nome da classe à qual o objeto pertence.
- C) Somente ao identificador usado para distinguir o objeto.
- D) Exclusivamente às mensagens recebidas de outros objetos.

**4. Sobre a relação entre classe e objeto, assinale a alternativa correta.**

- A) Classe e objeto são conceitos idênticos e podem ser utilizados sempre como sinônimos.
- B) Uma classe é uma ocorrência específica, enquanto o objeto representa a descrição geral.
- C) Uma classe descreve características e responsabilidades comuns, enquanto um objeto é uma instância específica com identidade e estado próprios.
- D) Um objeto só existe quando representa uma entidade física do mundo real.

**5. Em um sistema orientado a objetos, uma boa distribuição de responsabilidades ocorre quando:**

- A) Um único objeto central conhece todos os dados e executa todas as regras do sistema.
- B) Cada objeto assume comportamentos coerentes com as informações e responsabilidades que lhe pertencem.
- C) As classes armazenam somente dados e todos os comportamentos ficam fora delas.
- D) Todo substantivo presente na descrição do problema é transformado obrigatoriamente em uma classe.

**6. Questão aberta**

Considere um sistema de biblioteca no qual usuários podem retirar exemplares de livros, devolver exemplares e consultar a situação de seus empréstimos. Com base apenas nos fundamentos da Programação Orientada a Objetos estudados nesta semana, proponha uma modelagem conceitual inicial. Em sua resposta: (a) identifique pelo menos três classes candidatas; (b) dê um exemplo de objeto/instância; (c) descreva possíveis elementos de estado e comportamento de cada classe; e (d) explique ao menos uma colaboração entre os objetos. Justifique suas escolhas e evite elementos que não sejam relevantes para o domínio.

**Orientações para revisão**

Antes de avançar para a Semana 2, recomenda-se revisar os conceitos utilizando o seguinte encadeamento mental: paradigma → problema → objetos relevantes → classe e instância → identidade → estado → comportamento → responsabilidade → colaboração. O estudante deve conseguir explicar cada conceito com um exemplo próprio e justificar por que determinado elemento pertence ou não ao modelo.

**Referências de apoio**

- Plano de Ensino da disciplina 49297 - Programação Orientada a Objetos, Universidade de Uberaba (UNIUBE).
- SINTES, Anthony. Aprenda programação orientada a objetos em 21 dias. São Paulo: Pearson Education do Brasil, 2002.

- FÉLIX, Rafael. Programação Orientada a Objetos. Bibliografia indicada no plano de ensino institucional.
- BARNES, David J.; KÖLLING, Michael. Programação Orientada a Objetos com Java. Referência complementar do plano institucional; utilizar os capítulos conceituais de orientação a objetos, sem foco em sintaxe nesta etapa.

## GABARITO DO ROTEIRO DE ESTUDO 1

| Questão | Resposta | Conceito avaliado                 | Justificativa                                                                                                                       |
|---------|----------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1       | B        | Paradigma de programação          | Paradigma é uma forma de organizar o raciocínio e estruturar soluções conforme princípios. Não é uma linguagem específica.          |
| 2       | C        | Procedural × orientado a objetos  | A abordagem procedural tende a enfatizar passos e rotinas; a POO enfatiza entidades, responsabilidades e colaboração.               |
| 3       | A        | Estado do objeto                  | Estado é o conjunto de valores que descreve a situação atual de um objeto em determinado momento.                                   |
| 4       | C        | Classe × objeto                   | A classe descreve características e responsabilidades comuns; o objeto é uma instância específica com identidade e estado próprios. |
| 5       | B        | Distribuição de responsabilidades | Uma boa modelagem atribui a cada objeto comportamentos coerentes com aquilo que ele conhece e com a responsabilidade que assume.    |

Questão Aberta 6:

(a) **Classes candidatas:** Usuário, Livro, Exemplar e Empréstimo.

(b) **Exemplo de objeto/instância:**

exemplar101 : Exemplar, com código 101 e situação “disponível”.

(c) **Estado e comportamento:**

- **Usuário:** nome, matrícula, situação; pode consultar empréstimos.
- **Livro:** título, autor, ISBN; pode exibir informações.
- **Exemplar:** código, disponibilidade; pode ser emprestado e devolvido.
- **Empréstimo:** usuário, exemplar, datas, situação; pode ser realizado e finalizado.

(d) **Colaboração:**

O objeto **Empréstimo** associa um **Usuário** a um **Exemplar**. Ao realizar o empréstimo, verifica a disponibilidade do exemplar e altera sua situação para “emprestado”. Na devolução, o empréstimo é finalizado e o exemplar volta a ficar “disponível”.